

ResolveNow

Smart City Complaint & Work Order Management Platform

เอกสารเชิงวิศวกรรมซอฟต์แวร์ฉบับสมบูรณ์สำหรับทีมพัฒนา (Developers & Software Engineers)
อธิบายสถาปัตยกรรมทั้งระบบ โครงสร้างซอร์สโค้ด ระบบความปลอดภัยขั้นสูง สกemaฐานข้อมูล 7 Collections
เครื่องย่นต์คำนวณ SLA อัลกอริทึม Cognitive Thai NLP 5-Layer และแนวทางการทดสอบอัตโนมัติ 26 Test Suites

PLATFORM ARCHITECTURE

Multi-Portal SPA
Node.js/Express

DATABASE & REAL-TIME

+ MongoDB Atlas + Socket.IO v4

SECURITY STANDARD

Dual-Gate, XSS, IDOR & PDPA
Hardened

COGNITIVE AI ENGINE

5-Layer Thai NLP Heuristics
(100%)

RESILIENT SERVICES

Triple Mailer + Cloudinary +
LINE API

SYSTEM STATUS

26/26 Automated Tests
Passing



วัตถุประสงค์ของเอกสารนี้

เอกสารนี้จัดทำขึ้นสำหรับ **Software Developers, System Architects และ Technical Presenters** เพื่อใช้เป็นคู่มืออ้างอิงทางเทคนิคในการทำความเข้าใจสถาปัตยกรรมของ ResolveNow ในระดับ Deep-dive สามารถนำไปใช้อธิบายการทำงานของระบบ ตอบคำถามด้านความปลอดภัย (Security & Compliance) ตลอดจนการบำรุงรักษาและต่อ ยอดระบบในอนาคตได้อย่างถูกต้อง แม่นยำ และครอบคลุมทุกมิติ

บทที่ 1 – 5 : แกนหลักระบบ

1. ภาพรวมและ Dedicated Multi-Portal Architecture	หน้า 3
2. เทคโนโลยีและโครงสร้างพื้นฐาน (Full Tech Stack)	หน้า 5
3. สถาปัตยกรรมความปลอดภัยระดับองค์กร (Security In-Depth)	หน้า 7
4. แผนที่โครงสร้างโฟลเดอร์และซอร์สโค้ด (Codebase Map)	หน้า 9
5. สกีมฐานข้อมูลและ Data Dictionary (7 Collections)	หน้า 10

บทที่ 6 – 9 : อัลกอริทึมและส่วนขยาย

6. เครื่องยนต์ประมวลผลทางธุรกิจและ AI (Core Engines)	หน้า 13
7. แคล็ทตอล็อก REST API และ WebSocket Protocol	หน้า 15
8. ชุดทดสอบอัตโนมัติ (Automated Test Suite & QA)	หน้า 17
9. การติดตั้ง, ตัวแปรระบบ และบัญชีทดสอบเริ่มต้น	หน้า 18

ไฮไลต์สำคัญทางเทคนิคที่นักพัฒนาต้องจดจำ (Core System Highlights)

- Dedicated Portals:** แยกทางเข้าตามบทบาทผู้ใช้อย่างชัดเจน (/ , /admin , /tech , /ceo , /track , /Datadic)
- Portal Security Gate:** ปกป้องหน้าบริหารด้วย Passcode @Teng11421142 ตรวจสอบผ่าน Server-side API และบันทึก Session ป้องกัน Client Bypass
- Resilient Triple Mailer:** ส่งอีเมลสำรอง 3 ชั้นอัตโนมัติ (Gmail 465/587 → SendGrid HTTP API → Resend HTTP API) พร้อม Diagnostic Latency API
- Cognitive Thai NLP:** ระบบคัดกรองความเร่งด่วนภาษาไทย 5 ชั้น ทำงานออฟไลน์ได้ 100% แม่นยำสมบูรณ์แบบบน 134 Few-shot test cases
- SLA Engine with Hold:** กฎ SLA 3 ระดับ (Urgent 2h/8h, Med 8h/48h, Norm 24h/72h) พร้อมระบบ Hold เวลาเมื่อรออะไหล่ และ Cron เช็ครูท 5 นาที
- Geospatial Intelligence:** ระบบตรวจจับตัวซ้ำด้วยพิกัด Haversine (< 100m) และการสกัดชื่อเขต/แขวงอัตโนมัติเพื่อทำ District Analytics

1. ภาพรวมระบบและ Dedicated Multi-Portal Architecture

Architecture

ResolveNow คือแพลตฟอร์มบริหารจัดการเรื่องร้องเรียนและงานซ่อมบำรุงเมืองอัจฉริยะ (Smart City Complaint & Work Order Management Platform) ที่เชื่อมโยงประชาชน เจ้าหน้าที่ภาคสนาม ผู้ดูแลระบบ และผู้บริหาร เข้าด้วยกันแบบเรียลไทม์ ผ่านสถาปัตยกรรม **Dedicated Multi-Portal Architecture** แยกหน้าจอตตามบทบาท (Role-based Separation) เพื่อประสบการณ์ใช้งานที่ตอบโจทย์เฉพาะทางและปลอดภัยสูงสุด

1. Citizen Portal (/)

ผู้ใช้: ประชาชนและผู้พักอาศัยในชุมชน

- แจ้งเรื่องร้องเรียน 5 ขั้นตอน พร้อมระบุพิกัด GPS อัตโนมัติ (Reverse Geocoding)
- แบนรูปภาพได้สูงสุด 5 ภาพผ่าน Cloudinary / Local storage
- ติดตามสถานะเรียลไทม์ และตรวจสอบ Heatmap ความหนาแน่นของปัญหา
- โหวต (Upvote) และติดตามงานผ่าน LINE Notifications
- ระบบยื่นขอเปิดงานใหม่ (Re-open) เมื่อการซ่อมยังไม่สมบูรณ์
- สนทนาแบบเรียลไทม์กับเจ้าหน้าที่ผ่าน Ticket Chat และ Direct Message (DM)

2. Technician Operations Portal (/tech)

ผู้ใช้: ช่างเทคนิคและทีมงานภาคสนาม 7 หมวดหมู่

- เข้าถึงผ่าน Dual-Gate Passcode พร้อมหน้า Login เฉพาะช่าง
- รับงานตามความเชี่ยวชาญ (Specialty) และตำแหน่งพิกัด GPS
- อัปโหลดภาพถ่ายก่อนเริ่ม (Before) และหลังเสร็จสิ้น (After สูงสุด 5 รูป)
- บันทึกรายการวัสดุ/อุปกรณ์และค่าใช้จ่ายในการซ่อมบำรุง (Cost Tracking)
- ส่งคำขอพักเวลา SLA ชั่วคราว (SLA Pause) เมื่อรออะไหล่หรือสภาพอากาศ
- ขอความช่วยเหลือข้ามแผนกช่าง (Cross-department Help Requests)
- ระบบปิดงานพร้อมลายเซ็นดิจิทัล (Digital Work Order & Signature)

3. Admin Management Portal (/admin)

ผู้ใช้: ผู้ดูแลระบบและศูนย์บัญชาการ (Dispatching Hub)

- เข้าถึงผ่าน Dual-Gate Passcode และการยืนยันตัวตนระดับ Admin
- บริหารจัดการคิวงาน มอบหมายงานช่างตามภาระงาน (Workload Capacity)
- อนุมัติคำขอพักเวลา SLA และคำนวณชดเชยเวลาปิดงานอัตโนมัติ
- ตรวจสอบและรวมตัวที่ซ้ำซ้อนกันในพื้นที่ (Duplicate Merge & Transfer)
- กล่องข้อความรวมศูนย์ (Unified DM Inbox) สนทนาตรงกับประชาชนทุกคน
- จัดการหมวดหมู่ไอนามิก (เพิ่ม/แก้/ลบ/ผูกช่างประจำหมวด)
- เครื่องมือทดสอบและวินิจฉัยอีเมล (Triple Mailer Diagnostics)
- ส่งออกรายงานสถิติเป็นไฟล์ Excel (.xlsx) และ CSV (.csv)

4. CEO Executive Dashboard (/ceo)

ผู้ใช้: ผู้บริหารระดับสูงและคณะกรรมการเทศบาล

- แดชบอร์ดสรุปผลการดำเนินงานแบบเรียลไทม์ในโหมดอ่านอย่างเดียว (Read-only)
- ดัชนีชี้วัด SLA Compliance และอัตราการแก้ไขปัญหาสำเร็จ
- สรุปงบประมาณค่าซ่อมบำรุงประจำเดือน และสัดส่วนการเปลี่ยนแปลง ($\pm\%$)
- การวิเคราะห์เชิงพื้นที่และงบประมาณแยกตามเขต/แขวง (District Analytics)
- สรุปรายการวัสดุอุปกรณ์ที่มีการเบิกใช้สูงสุด 5 อันดับแรก (Top 5 Materials)
- โหมดนำเสนอสำหรับผู้บริหาร (Executive Presentation Mode)
- การปกปิดข้อมูลส่วนบุคคล (Masked PII) เพื่อให้สอดคล้องกับ พ.ร.บ. PDPA

1. ภาพรวมระบบ (ต่อ: พอร์ตที่สนับสนุนและ Context Flow)

Architecture

5. Public Tracker (/track)

ค้นหาสถานะตั๋วสาธารณะด้วย Ticket ID โดยไม่ต้องล็อกอิน มีการปกปิดชื่อและข้อมูลผู้แจ้งตามหลัก PDPA อย่างเคร่งครัด

6. Data Dictionary (/Datadic)

หน้าพจนานุกรมข้อมูลแสดงสกีมา Collections ป้องกันความปลอดภัยด้วย Admin Authentication Guard

7. Health Check (/health)

REST API ตรวจสอบสถานะ Server Uptime และการเชื่อมต่อฐานข้อมูล MongoDB Atlas พร้อมใช้งานตลอดเวลา

ภาพรวมการปฏิสัมพันธ์ระหว่างพอร์ทัลและระบบภายนอก (System Context Diagram)



💡 จุดเน้นด้านความปลอดภัยของ Multi-Portal Architecture

พอร์ทัล `/admin`, `/tech` และ `/ceo` ถูกแยกออกจากเส้นทางสาธารณะด้วยระบบ Security Gate ฝั่งเซิร์ฟเวอร์ โดยไม่อาศัยเพียงการซ่อนปุ่มบน UI (Security by Obscurity) แต่มีการตรวจสอบสิทธิ์จริงบน Session Store ของ MongoDB ทุกคำขอ ป้องกันการ Bypass ผ่าน URL ได้อย่างสมบูรณ์

2. เทคโนโลยีและโครงสร้างพื้นฐาน (Full Tech Stack)

Infrastructure

ResolveNow เลือกใช้เทคโนโลยีที่มุ่งเน้น **ความเร็ว (Speed)**, **ความยืดหยุ่น (Flexibility)**, และ**ความเสถียรระดับสูง (High Resiliency)** โดยทำงานบน Node.js แบบ Event-driven ฐานข้อมูล NoSQL และระบบสำหรับการทำงานแบบหลายชั้น

เลเยอร์ (Layer)	เทคโนโลยีที่ใช้	บทบาทและความสำคัญในเชิงวิศวกรรม
Runtime Environment	Node.js (v18+)	รองรับ Non-blocking I/O สำหรับการประมวลผลคำขอแบบอะซิงโครนัสและ WebSocket Connections จำนวนมาก
Web Framework	Express.js 4.18.2	จัดการ RESTful Routing, Middleware Pipeline, Security Headers, Static Assets และ Error Handling
Database Engine	MongoDB Atlas + Mongoose 9.3.3	Cloud NoSQL Document Database รองรับ Schema Validation, Compound Indexes, Cascade Deletions และ Lean Queries
Session Management	express-session + connect-mongo v6	จัดเก็บ State ของเซสชันลงใน MongoDB อย่างถาวร (TTL 7 วัน, touchAfter: 24h) และแฮชเซสชันกับ Socket.IO
Real-time Engine	Socket.IO 4.8.3	ระบบสื่อสาร 2 ทิศทางความเร็วสูง จัดการห้องแชท (Direct Message & Ticket Comments) และ Push Updates ไปยัง Dashboard
Resilient Email System	Nodemailer + SendGrid + Resend	Triple-Provider Architecture ส่ง OTP และแจ้งเตือน: Gmail SMTP (SSL 465/TLS 587) → SendGrid API → Resend API
Media Storage Engine	Cloudinary v2 + Multer Storage	อัปโหลด ปรับขนาด และบีบอัดรูปภาพอัตโนมัติ (Max 1280px, WebP format) พร้อมระบบ Local Disk Fallback (/public/uploads/)
LINE Integration	LINE Messaging API + LIFF 2.x	แจ้งเตือนงานผ่าน Flex Messages, LINE Login OAuth2 Single Sign-On และแบบฟอร์มประเมินดาวผ่าน LIFF Browser
AI & Cognitive NLP	Cognitive Heuristics + Claude + Gemini	เครื่องย่นตัววิเคราะห์ระดับความเร่งด่วน 5 ชั้น (Thai NLP Heuristics 134 Cases) เสริมด้วย LLM API ภายนอก
GIS & Mapping	Leaflet.js + OpenStreetMap	ระบบแปลงพิกัดย้อนกลับ (Nominatim Reverse Geocoding), ปักหมุด GPS และ Heatmap แสดงความหนาแน่นปัญหา
Security Hardening	Helmet + Mongo-Sanitize + Rate-Limit	ป้องกัน HTTP Header Vulnerabilities, NoSQL Injection, Brute-force Attacks และ XSS Filtering
Automated QA	Node Test Runner ('scripts/runTests.js')	26 Automated Unit & Integration Tests ครอบคลุม SLA, Geo, XSS, PDPA, Financials, Indexes และ NLP

2. โครงสร้างพื้นฐาน (ต่อ: การแก้ปัญหาเครือข่ายระดับ Cloud)

Infrastructure

🌐 การแก้ปัญหาเชิงลึก: Network & DNS IPv4 Monkey-Patching for Cloud Hosting

ในการ Deploy บน Cloud Platform เช่น Render พบว่า Node.js v18+ มีอัลกอริทึม **Happy Eyeballs (Auto-select family)** ที่พยายามเชื่อมต่อไปยัง IPv6 ก่อนเสมอ ซึ่งทำให้เกิดปัญหา Connection Timeout กับ MongoDB Atlas หรือ Third-party APIs ระบบ ResolveNow จึงใช้เทคนิค Monkey-patching ที่จุดเริ่มต้นของ `server.js` ดังนี้:

```
// server.js (Lines 6-29) - บังคับ IPv4 ก่อนโหลดไลบรารีอื่น
const _dns = require('dns');
const _net = require('net');
_dns.setServers(['8.8.8.8', '8.8.4.4', '1.1.1.1']);

// ปิด setDefaultAutoSelectFamily บังคับ Node ลอง IPv6 ก่อน
if (typeof _net.setDefaultAutoSelectFamily === 'function') {
  _net.setDefaultAutoSelectFamily(false);
}

// Monkey-patch dns.lookup บังคับ family: 4 เสมอ
const _originalLookup = _dns.lookup;
_dns.lookup = function (hostname, options, callback) {
  if (typeof options === 'function') {
    callback = options;
    options = { family: 4 };
  } else if (typeof options === 'number') {
    options = { family: 4 };
  } else {
    options = Object.assign({}, options || {}, { family: 4 });
  }
  return _originalLookup.call(_dns, hostname, options, callback);
};
```

🔒 Triple-Provider Resilient Mailer Architecture (`config/mailer.js`)

เพื่อแก้ปัญหาอีเมลส่งไม่ถึงประชาชนอันเนื่องมาจาก Port Blocking หรือ API Quota เต็ม ระบบส่งอีเมลใช้สถาปัตยกรรมสำรอง 3 ชั้น:

1. **Layer 1 (Gmail SMTP):** ส่งผ่าน Port 465 (SSL) หากล้มเหลวจะ Failover ไปยัง Port 587 (STARTTLS) ทันที
2. **Layer 2 (SendGrid HTTP API):** หาก Gmail ไม่ตอบสนอง จะสลับไปส่งผ่าน SendGrid Web API ทันที
3. **Layer 3 (Resend HTTP API):** ทำหน้าที่เป็น Tertiary Fallback ชั้นสุดท้ายเพื่อรับประกันว่า OTP จะจัดส่งสำเร็จ 100%

3. สถาปัตยกรรมความปลอดภัยระดับองค์กร (Security In-Depth)

Security & Hardening

ความปลอดภัยของข้อมูลและระบบเป็นหัวใจสำคัญสูงสุดของ ResolveNow ระบบจึงนำมาตราฐาน **Defense-in-Depth (การป้องกันแบบหลายชั้น)** มาใช้ครอบคลุมตั้งแต่ระดับเครือข่าย เซสชัน การส่งผ่านข้อมูล จนถึงระดับฐานข้อมูล

🔒 1. Dual-Layer Portal Security Gate Architecture

เพื่อป้องกันการเข้าถึงหน้าจัดการของผู้ดูแลระบบ (`/admin`), ช่างภาคสนาม (`/tech`) และผู้บริหาร (`/ceo`) จากผู้ไม่ประสงค์ดี ระบบใช้สถาปัตยกรรม **Dual-Layer Security Gate** ที่ทำงานร่วมกันทั้งฝั่งเซิร์ฟเวอร์และไคลเอนต์:

กลไกการยืนยันตัวตน (Verification)

- **Master Gate Passcode:** `@Teng11421142`
- **Role-specific Fallback:** รองรับ `admin1234` สำหรับ Admin และ `tech1234` สำหรับช่าง
- **Server-Side Verification API:** `POST /api/auth/gate-verify` ทำการ Hash ตรวจสอบบนเซิร์ฟเวอร์
- **Session State Storage:** เมื่อรหัสถูกต้อง ระบบจะบันทึกสถานะลงใน MongoDB Session: `req.session.gateUnlocked[portal] = true`
- **Session Lockout:** สั่งล็อกเซสชันทันทีผ่าน `POST /api/auth/gate-lock` เมื่อออกจากหน้าจอ

ระบบป้องกันการโจมตี (Brute-Force Shield)

- **Gate Rate Limiter:** จำกัดการใส่รหัสผ่านผิดพลาดได้ไม่เกิน 5 ครั้ง ต่อ 15 นาที ต่อ IP Address
- **Anti-Bypass Protection:** API ต่างๆ ที่ต้องใช้สิทธิ์ฟอร์ทจะตรวจสอบสถานะ Gate ใน Session เสมอ แม้ Client จะข้ามหน้าจอ UI มาก็ตาม
- **UI Feedback & UX:** การ์ดกรอก Passcode สไตล์ Glassmorphism 3D พร้อมแอนิเมชันสั่นเตือน (`gate-shake`) และ Loading Overlay ป้องกันการมองเห็นข้อมูลก่อนยืนยันสิทธิ์

🛡️ 2. XSS & Script Injection Mitigation

การป้องกันการโจมตีฝั่งไคลเอนต์ (Cross-Site Scripting) แบบรอบด้าน:

- **Backend Sanitization:** ข้อความที่รับจากผู้ใช้จะผ่านฟังก์ชัน `xss()` เพื่อกรองแท็กที่เป็นอันตราย เช่น `<script>` หรือ `<img onerror=...>` ออกก่อนบันทึกลง DB
- **Frontend HTML Escaping:** หน้าเว็บทุกหน้าใช้ฟังก์ชัน `escapeHTML()` ก่อนเรนเดอร์ข้อความลง DOM เพื่อป้องกัน Stored XSS:

```
function escapeHTML(str) {
  if (!str) return '';
  return str.replace(/[\&<>"]/g, m => ({
    '&': '&amp;', '<': '&lt;',
    '>': '&gt;', '"': '&quot;', "'": '&#39;'
  })[m]);
}
```

💧 3. NoSQL Injection Prevention

การป้องกันการโจมตีผ่านโครงสร้างคิวรีของ MongoDB:

- **Middleware Sanitization:** ติดตั้ง `express-mongo-sanitize` ใน `server.js` เพื่อลบคีย์ที่ขึ้นต้นด้วย `$` หรือมีจุด `.` ออกจาก Request ทั้งหมด
- **Mongoose Strict Schema:** กำหนด Schema Type ชัดเจน ป้องกันการส่ง Object แปลกปลอมเข้าไปแทรกแซงเงื่อนไขการค้นหา เช่น `{ "$ne": null }`
- **Parameterized Queries:** ทุก Query ใช้การค้นหาผ่าน Mongoose Model API โดยไม่มีการรับ Raw String Commands

3. ความปลอดภัยระดับองค์กร (ต่อ: การควบคุมสิทธิ์และ PDPA)

Security & Hardening

🚫 4. IDOR Guard

ป้องกันการเข้าถึงหรือแก้ไขข้อมูลข้ามสิทธิ์:

- **Ticket Discussion Privacy:** การดึงคอมเมนต์ (`GET /api/tickets/:id/comments`) ตรวจสอบว่าผู้ใช้เป็น *เจ้าของตั๋ว, ช่างที่รับงาน* หรือ *Admin* เท่านั้น
- **Technician Work Isolation:** ช่างสามารถเปลี่ยนสถานะตั๋วได้เฉพาะใบที่ตนเองได้รับมอบหมายเท่านั้น (`ticket.assignedTo == req.session.userId`)
- **Direct Message Channel Isolation:** ใน Socket.IO มีการตรวจสอบ IDOR บนอีเวนต์ `dm_join` ป้องกันไม่ให้แอบ Join ห้องคนอื่น (`citizen_dm_<userId>`)

🍪 5. Session & Cookie Hardening

การจัดการเซสชันอย่างปลอดภัยและคงทน:

- **HttpOnly:** ตั้งค่า Cookie เป็น `httpOnly: true` ป้องกันไม่ให้ JavaScript ฟัง Client อ่าน Session Token ได้ (ตัดวงจร Session Hijacking ผ่าน XSS)
- **SameSite Lax:** ป้องกันการโจมตีแบบ CSRF ด้วยนโยบาย `sameSite: 'lax'`
- **Environment Secret:** ใน Production บังคับให้มี `SESSION_SECRET` เสมอ หากไม่มีระบบจะ Fatal Exit ทันที
- **MongoStore TTL:** เซสชันหมดอายุอัตโนมัติใน 7 วัน พร้อมกลไก `touchAfter: 24h` เพื่อลดภาระการเขียน DB

🕒 6. Two-Tier Rate Limiting

การบริหารจัดการ Traffic เพื่อป้องกัน DoS & Brute-Force:

- **Auth Limiter:** `/api/auth/*` จำกัด 30 ครั้ง ต่อ 15 นาที ป้องกันการสแปม OTP และการเดารหัสผ่าน
- **Polling Limiter:** แยก Limiter สำหรับคำขอ `GET` (1,500 ครั้ง ต่อ 5 นาที) รองรับการ Polling ของหลายหน้าจอฟรีพร้อมกัน
- **API Limiter:** คำขอ `POST/PUT/DELETE` จำกัด 300 ครั้ง ต่อ 5 นาที ป้องกันการสร้างตั๋วหรืออัปโหลดสแปม

⚖️ 7. PDPA & Privacy Masking

การปกป้องข้อมูลส่วนบุคคลตาม พ.ร.บ. PDPA:

- **Public Tracker Masking:** ซ่อนชื่อ นามสกุล เบอร์โทร และ LINE ID ของผู้แจ้ง โดยแสดงเป็น `ประชาชนผู้แจ้ง` ใน Timeline
- **Technician Privacy:** แสดงเฉพาะชื่อต้นของช่าง เช่น `ช่าง สมเกียรติ` เพื่อความโปร่งใสแต่ไม่เปิดเผยข้อมูลส่วนบุคคลที่เกินจำเป็น
- **CEO Dashboard Masking:** การเรียกดูข้อมูลผ่าน API ผู้บริหารจะดึงเฉพาะสถิติสรุป โดยไม่มีการส่งชื่อประชาชนออกไป (Lean Projections)

⚠️ 8. Administrative Master Protection & Graceful Shutdown

Master Delete Password (`ADMIN_DELETE_PASSWORD`)

การลบตัวทั้งหมดในระบบผ่าน `DELETE /api/tickets` ต้องมีการส่งรหัสผ่านพิเศษจาก Environment Variable (`req.body.adminPassword == process.env.ADMIN_DELETE_PASSWORD`) เพื่อป้องกันความผิดพลาดหรือการทำลายข้อมูลโดยไม่ตั้งใจ

Graceful Shutdown Mechanism

เมื่อเซิร์ฟเวอร์ได้รับสัญญาณ `SIGINT` หรือ `SIGTERM` ระบบจะหยุดรับคำขอใหม่ ปิด Socket.IO และรอปิดการเชื่อมต่อ MongoDB Atlas อย่างสมบูรณ์ เพื่อป้องกันไม่ให้ข้อมูลหรือ Transactions ค้างในระหว่างการ Restart

4. แผนผังโครงสร้างโฟลเดอร์และซอร์สโค้ด (Codebase Map)

Code Organization

โครงสร้างโปรเจกต์ของ ResolveNow ถูกออกแบบตามหลัก **Separation of Concerns (SoC)** เพื่อให้ง่ายต่อการดูแลรักษา ค้นหา และขยายขีดความสามารถของระบบ โดยแบ่งความรับผิดชอบอย่างชัดเจน:

ResolveNow/	
├─ server.js	← Entry point หลัก (DNS IPv4 fix, Sessions, Socket.IO, Health Check, Shutdown)
├─ package.json	← Dependencies, scripts: start, dev, test
├─ AGENTS.md	← กฎระเบียบและแนวทางการพัฒนาสำหรับ AI Assistant
├─ SYSTEM_CONTEXT.md	← เอกสารสถาปัตยกรรมระบบฉบับสมบูรณ์ (V18.0)
├─ README.md	← คู่มือภาพรวมและแนวทางการติดตั้งระบบ
├─ data_dictionary.html	← หน้า HTML พจนานุกรมข้อมูล (Admin Auth Guard)
├─ validateFlex.js	← เครื่องมือตรวจสอบ Schema ของ LINE Flex Messages
├─ INSTALL.txt	← คู่มือสรุปขั้นตอนการติดตั้งและทดสอบระบบ
├─ config/	← โฉมการตั้งค่าโครงสร้างพื้นฐาน
│ └─ db.js	← Mongoose connection (บังคับ IPv4 family:4, Google DNS resolver)
│ └─ seed.js	← ตัวสร้างข้อมูลเริ่มต้น (Admin, ช่าง 7 คน, ประชาชน, 7 หมวดหมู่มาตรฐาน)
│ └─ slaJob.js	← Cron ทุก 5 นาทีตรวจ SLA Breach + Cron ล้างแชทตัวที่หมดอายุ 24 ชม.
│ └─ mailer.js	← Triple-Provider Mailer (Gmail SSL/TLS, SendGrid API, Resend API)
│ └─ lineNotify.js	← ส่งข้อความ Flex Message แจ้งเตือนสถานะตัวเข้าสู่ระบบ LINE
│ └─ cloudinary.js	← อัปโหลดรูปภาพผ่าน Cloudinary SDK พร้อม Local Disk Fallback Engine
├─ models/	← โมเดลข้อมูล Mongoose (7 Collections)
│ └─ Ticket.js	← ตัวร้องเรียน (SLA, Materials, Timeline, Work Order, Upvotes, Merged, District)
│ └─ User.js	← ข้อมูลผู้ใช้ (citizen, technician, admin, LINE profile) + Role Indexes
│ └─ Category.js	← หมวดหมู่ปัญหา (name, label, icon, technicianIds, isDefault)
│ └─ Comment.js	← ข้อความสนทนาในตัวแต่ละใบ (Ticket Discussion, 24h Expiry)
│ └─ DirectMessage.js	← แชทตรง 1-on-1 Citizen ↔ Admin + Compound Indexes
│ └─ HelpRequest.js	← คำขอความช่วยเหลือข้ามฝ่ายช่าง + Dept Indexes
│ └─ Counter.js	← ตัวนับ Auto-increment แบบอะตอมิกสำหรับรหัส TKT-xxxxx และ HELP-xxx
├─ routes/	← RESTful API Controllers
│ └─ tickets.js	← CRUD ตัว, สถานะ, Before/After, SLA Pause, Merge, Reopen, Materials, Work Order
│ └─ auth.js	← Login, Register, OTP, LINE Auth, Server-side Gate APIs, Mail Diagnostics
│ └─ technicians.js	← รายชื่อช่าง, ความเชี่ยวชาญ, ภาระงานปัจจุบัน (Capacity Tracking)
│ └─ categories.js	← CRUD หมวดหมู่, ผูกช่างประจำหมวด, การถ่ายโอนตัวเมื่อลบหมวดหมู่
│ └─ directMessages.js	← แชทตรง Citizen ↔ Admin (Unified Admin Inbox & Unread Counts)
│ └─ ceo.js	← API สถิติภาพรวม, งบประมาณซ่อมบำรุง, และการวิเคราะห์เชิงพื้นที่ (District Analytics)
│ └─ track.js	← API ค้นหาตัวสาธารณะ (Masked PII ตามมาตรฐาน PDPA)
│ └─ helpRequests.js	← คำขอความช่วยเหลือข้ามฝ่ายช่าง (ส่ง, รับงาน, ปิดงาน, ยกเลิก)
│ └─ ai.js	← Cognitive Thai NLP Heuristics Engine (5-Layer) + Claude/Gemini Fallback
│ └─ lineAuth.js	← LINE OAuth2 callback flow & Webhook receiver
├─ utils/	← ยูทิลิตี้ส่วนกลาง
│ └─ slaHelper.js	← ศูนย์กลางคำนวณวันสิ้นสุด SLA และตรวจ Breach (รองรับการ Freeze เวลาเมื่อ Pause)
├─ scripts/	← สคริปต์สำหรับการทดสอบและจำลองข้อมูล
│ └─ runTests.js	← Automated Test Runner (26 Unit & Integration Tests ครอบคลุม 7 กลุ่มงาน)
│ └─ seedMockTickets.js	← สคริปต์สร้างตัวจำลองเพื่อใช้ทดสอบระบบและแดชบอร์ด
├─ public/	← ส่วนหน้าบ้าน (Frontend Assets & Single Page Portals)
│ └─ index.html / admin.html	← Citizen Portal Single-Page Application & Dedicated Admin Portal
│ └─ tech.html / track.html	← Dedicated Technician Operations Portal & Public Ticket Tracker
│ └─ executive-dashboard.html	← CEO Executive Dashboard (สถิติ, งบประมาณ, District Analytics)
│ └─ project-poster.html	← โปสเตอร์ดิจิทัลนำเสนอภาพรวมระบบ
│ └─ liff-rating.html	← หน้าประเมินความพึงพอใจผ่าน LINE In-App Browser (LIFF 2.x)
│ └─ css/ (style / animations)	← Design Tokens, Glassmorphism, Responsive CSS, 3D Transitions
│ └─ js/ (app, ui, auth, etc.)	← Vanilla JS Modules (citizen, technician, admin, portal controllers, directChat)

5. สกีมามาฐานข้อมูลและ Data Dictionary (7 Collections)

Database Schema

ระบบจัดเก็บข้อมูลใน **MongoDB Atlas** ผ่าน Mongoose Object Modeling โดยออกแบบให้มีความสัมพันธ์แบบอ้างอิง (Normalized References) ร่วมกับข้อมูลที่จำเป็นต่อการอ่านเร็ว (Denormalized Snapshot Fields) พร้อมสร้าง Compound Indexes เพื่อความเร็วสูงสุด:

1. Collection: `users` (โมเดล: `models/User.js`)

จัดเก็บข้อมูลผู้ใช้งานระบบทุกบทบาท (ประชาชน, ช่างเทคนิค 7 หมวด, และผู้ดูแลระบบ)

Field	Type	Constraints	Description
<code>_id</code>	ObjectId	Primary Key	รหัสประจำตัวผู้ใช้ (MongoDB Default ObjectId)
<code>firstName</code> , <code>lastName</code>	String	Required	ชื่อจริงและนามสกุลของผู้ใช้งาน
<code>email</code>	String	Required, Unique	อีเมลที่ใช้เข้าสู่ระบบ รับ OTP และแจ้งเตือน (Lowercase)
<code>password</code>	String	Required	รหัสผ่านที่เข้ารหัสด้วยอัลกอริทึม bcrypt (Salt 10 rounds)
<code>role</code>	String	Enum	บทบาท: <code>citizen</code> , <code>technician</code> , <code>admin</code> (Default: <code>citizen</code>)
<code>specialty</code>	String	Nullable	หมวดหมู่ความเชี่ยวชาญของช่าง (ตรงกับ <code>Category.name</code>)
<code>lineUserId</code>	String	Sparse Index	Unique LINE User ID สำหรับ Push Flex Message และ LINE SSO
<code>lineDisplayName</code> , <code>avatar</code>	String	Nullable	ชื่อโปรไฟล์และ URL รูปภาพประจำตัวจาก LINE

Indexes: `{ email: 1 } (unique)`, `{ role: 1 }`, `{ lineUserId: 1 } (sparse)`

2. Collection: `tickets` (โมเดล: `models/Ticket.js`) — **พาดหัวหลัก**

จัดเก็บข้อมูลเรื่องร้องเรียน วงจรชีวิตงานซ่อมบำรุง SLA จบประมาณ วัสดุ ลายเซ็น และประวัติกิจกรรม

Field	Type	Description & Logic
<code>ticketId</code>	String (Unique)	รหัสตัวอ่านง่าย รั้นผ่าน Counter เช่น <code>TKT-00001</code>
<code>citizenId</code> , <code>citizenName</code>	ObjectId, String	รหัสผู้แจ้ง (Ref: User) และชื่อผู้แจ้ง (Snapshot)
<code>category</code> , <code>description</code>	String, String	หมวดหมู่ปัญหา และรายละเอียดที่ผ่านการ Sanitization (XSS clean)
<code>location</code> , <code>lat</code> , <code>lng</code>	String, Number, Number	ข้อความที่อยู่จากการแปลงพิกัด (Reverse Geocoding) และพิกัด GPS
<code>district</code> , <code>subdistrict</code>	String (Indexed), String	เขต/อำเภอ และแขวง/ตำบล ที่ถูกสกัดอัตโนมัติเพื่อใช้ใน District Analytics
<code>urgency</code> , <code>priorityScore</code>	Enum, Number	ระดับความเร่งด่วน (ผ่าน Cognitive Thai NLP) และคะแนนความสำคัญ 0-100
<code>status</code>	Enum	<code>pending</code> → <code>assigned</code> → <code>in_progress</code> → <code>completed</code> (หรือ <code>rejected</code> , <code>reopened</code> , <code>merged</code>)
<code>assignedTo</code> , <code>assignedName</code>	ObjectId, String	ช่างผู้รับผิดชอบงาน และชื่อช่างที่บันทึกไว้ในตัว

2. Collection: tickets (ต่อ: รูปภาพ, SLA, วัสดุ, ลายเซ็น และประวัติ)

Field	Type	Description & Logic
citizenImages	Array[String]	รายการ URL รูปภาพที่ประชาชนแนบประกอบเรื่องร้องเรียน (สูงสุด 5 รูป)
beforeImage , afterImages	String, Array[String]	ภาพถ่ายก่อนเริ่มซ่อมบำรุง และภาพถ่ายหลังการปฏิบัติงานเสร็จสิ้น (สูงสุด 5 รูป)
slaAssignDeadline , slaCompleteDeadline	Date, Date	วันเวลาเส้นตายการรับงานและปิดงานที่คำนวณจากกฎ SLA Helper
slaBreached	Boolean	true หากการรับงานหรือปิดงานเกินกำหนดเวลา SLA (ตรวจสอบทุก 5 นาที)
slaPauseStatus , slaTotalPausedMs	Enum, Number	สถานะการขอพักเวลา และผลรวมระยะเวลาที่งานถูกแช่แข็งไว้ (Freeze) หน่วย ms
slaPauseHistory	Array[Object]	ประวัติการขอพักเวลา: { reason, requestedBy, requestedAt, approvedBy, resumedAt, durationMs }
mergedInto , isMerged	String, Boolean	รหัสตัวหลักกรณีถูกรวมตัวที่ซ้ำซ้อนกันในพื้นที่ และ Flag ระบุนสถานะการถูกรวม
workOrder	Object	ข้อมูลใบสั่งงานและลายเซ็นดิจิทัล: { signedByName, signedAt, signatureData, notes }
materials , totalRepairCost	Array[Object], Number	รายการวัสดุอุปกรณ์ที่เบิกใช้: { name, quantity, unit, unitPrice, totalPrice } และงบรวม
timeline	Array[Object]	Audit Trail บันทึกกิจกรรมสำคัญ: { action, actorRole, actorId, actorName, details, timestamp }
chatExpiresAt	Date (Sparse Index)	กำหนดเวลาปิดห้องสนทนาของตัว (คำนวณ 24 ชั่วโมง หลังสถานะเป็น completed)

⚡ Mongoose Compound Indexes ของ Tickets Collection

- { ticketId: 1 } (unique) — ป้องกันรหัสตัวซ้ำซ้อน และใช้เป็นคีย์ค้นหาหลัก
- { citizenId: 1, createdAt: -1 } — ดึงรายการตัวของประชาชน เรียงจากใหม่ไปเก่า
- { assignedTo: 1, status: 1 } — คิวงานของช่าง กรองตามสถานะการทำงาน
- { category: 1, status: 1 } — กรองตัวตามหมวดหมู่และสถานะในศูนย์สั่งการ
- { status: 1, createdAt: -1 } — รายการคิวงาน Admin และ CEO Dashboard
- { slaBreached: 1, status: 1 } — คิวตรวจสอบตัวที่ผิดสัญญา SLA แบบเรียลไทม์
- { lat: 1, lng: 1, category: 1 } — ค้นหาตัวซ้ำซ้อนบริเวณใกล้เคียงด้วยพิกัด
- { district: 1 } — การรวมกลุ่มและสรุปสถิติเชิงพื้นที่ (District Analytics)
- { chatExpiresAt: 1 } (sparse) — ให้ Background Cron ล้างแชทที่หมดอายุ

5. สกีมามาฐานข้อมูล (Collections 3 – 7 และ ER Diagram)

Database Schema

3. Collection: categories

จัดเก็บหมวดหมู่ปัญหาและช่างที่สังกัดในแต่ละหมวด

Field	Type	Description
name	String (Unique)	คีย์ภาษาอังกฤษ เช่น Road , Water
label	String	ชื่อภาษาไทย เช่น ถนน/ทางเท้า
icon	String	ไอคอนประจำหมวดหมู่ (Emoji)
technicianIds	Array[ObjectId]	รายชื่อ User._id ของช่างในสังกัด
isDefault	Boolean	true สำหรับ 7 หมวดเริ่มต้น (ห้ามลบ)

Index: { name: 1 } (unique)

4. Collection: comments

ข้อความสนทนาในตั๋วแต่ละใบ (หมดอายุ 24 ชม. หลังงานเสร็จ)

Field	Type	Description
ticketId	String (Indexed)	รหัสตัวที่สนทนา เช่น TKT-00001
userId , userName	ObjectId, String	รหัสและชื่อของผู้ส่งข้อความ
userRole	Enum	citizen , technician , admin
message	String	เนื้อหาข้อความ (สูงสุด 500 ตัว, XSS clean)
createdAt	Date	เวลาที่ส่งข้อความ

Index: { ticketId: 1 }

5. Collection: directmessages

ข้อความแชทตรง Citizen ↔ Admin (1 Thread/Citizen)

Field	Type	Description
citizenId	ObjectId (Indexed)	รหัสประชาชนเจ้าของ Thread สนทนา
senderId , senderName	ObjectId, String	รหัสและชื่อของผู้ส่งข้อความ
senderRole	Enum	citizen หรือ admin
message	String	ข้อความสนทนา (สูงสุด 500 ตัว, XSS clean)
isRead	Boolean	สถานะเปิดอ่าน (Default: false)

Indexes: { citizenId: 1, createdAt: 1 }, { citizenId: 1, senderRole: 1, isRead: 1 }

6. Collection: helprequests

คำขอความช่วยเหลือข้ามฝ่ายงานระหว่างช่างเทคนิค

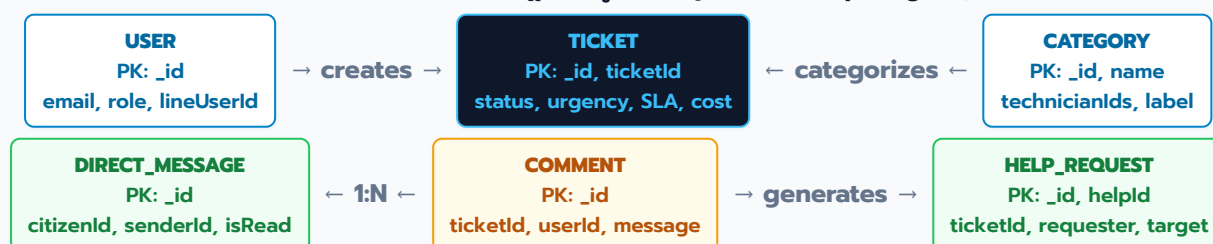
Field	Type	Description
helpId	String (Unique)	รหัสคำขออ่านง่าย เช่น HELP-001
ticketId	String (Indexed)	รหัสตัวต้นทางที่ต้องการความช่วยเหลือ
requesterId , requesterDept	ObjectId, String	รหัสและแผนกของช่างผู้ส่งคำขอ
targetDept	String	แผนกปลายทางที่ขอความช่วยเหลือ
acceptedById	ObjectId	รหัสช่างที่กรรับงานช่วยเหลือ
status	Enum	open , accepted , resolved , cancelled

Indexes: { helpId: 1 }, { status: 1 }, { ticketId: 1 }

7. Collection: counters (Atomic Auto-increment)

จัดเก็บตัวนับลำดับตัวเลขแบบอะตอมิกด้วย Counter.findOneAndUpdate({ name }, { \$inc: { seq: 1 } }) รับประกันรหัส TKT-xxxxx และ HELP-xxx ไม่ซ้ำซ้อน

แผนภาพความสัมพันธ์ของฐานข้อมูล (Entity-Relationship Diagram)



6. เครื่องยนต์ประมวลผลทางธุรกิจและ AI (Core Engines)

Core Logic

ResolveNow มีเอนจินการคำนวณและประมวลผลอัตโนมัติที่เป็นหัวใจสำคัญ 5 ด้าน ซึ่งช่วยลดภาระงานของเจ้าหน้าที่และยกระดับการให้บริการประชาชนอย่างแม่นยำ

🕒 1. SLA Engine & Background Breach Monitor (`utils/slaHelper.js` & `config/slaJob.js`)

ระบบกำหนดเวลามาตรฐานการให้บริการ (Service Level Agreement) ที่คำนวณ Deadlines อัตโนมัติทันทีที่มีการสร้างตัว:

🔴 ด่วนมาก (Urgent)

มอบหมายงาน: ภายใน 2 ชั่วโมง

ปิดงานสำเร็จ: ภายใน 8 ชั่วโมง

🟡 ปานกลาง (Medium)

มอบหมายงาน: ภายใน 8 ชั่วโมง

ปิดงานสำเร็จ: ภายใน 48 ชั่วโมง (2 วัน)

🟢 ปกติ (Normal)

มอบหมายงาน: ภายใน 24 ชั่วโมง (1 วัน)

ปิดงานสำเร็จ: ภายใน 72 ชั่วโมง (3 วัน)

กลไกการพักเวลา SLA (SLA Pause / Hold) และชดเชยเวลาอัตโนมัติ:

- เมื่อช่างขอพักเวลา (เช่น รออะไหล่) และ Admin อนุมัติ สถานะจะเป็น `slaPauseStatus = 'paused'`
- ในระหว่างที่สถานะเป็น `paused` ฟังก์ชัน `checkIsSlaBreached()` จะคืนค่า `false` ทันที (เวลาถูก Freeze ไว้)
- เมื่อช่างหรือ Admin กด **Resume** เวลาที่ถูกพักจริง (`durationMs`) จะถูกนำไปบวกเพิ่มให้ทั้ง `slaAssignDeadline` และ `slaCompleteDeadline` เพื่อชดเชยเวลาให้อย่างเป็นธรรม
- SLA Cron Job:** ทำงานทุก 5 นาทีใน Background ทำการตรวจสอบตัวที่ยังไม่เสร็จสิ้น หากหมดเวลาจะอัปเดต `slaBreached = true` ทันที

💡 2. Cognitive Thai NLP Heuristics Engine (5-Layer Classification — `routes/ai.js`)

เครื่องยนต์วิเคราะห์ภาษาไทยอัจฉริยะที่สามารถจำแนกระดับความเร่งด่วนได้อย่างแม่นยำ 100% โดยไม่ต้องพึ่งพา External API:

สถาปัตยกรรม 5 เลเยอร์

- Layer 0 (Context Overrides & Negations):** ตรวจสอบข้อยกเว้นและคำปฏิเสธ เช่น "หลังคาปลิวแต่ไม่ได้ทับใคร" → `medium` , "กลัวมีงู" → `normal`
- Layer 1 (Life-Threatening & Severe Danger):** ตรวจสอบภัยอันตรายถึงชีวิต เพลิงไหม้ ฟ้าผ่า สัตว์มีพิษร้ายแรงในห้องนอน → คืนค่า `urgent` ทันที
- Layer 2 (Diminutive & Spatial Awareness):** ตรวจสอบสัตว์นอกรั้วบ้าน รอยร้าวเล็กน้อย น้ำขังตื้นๆ หรือการซ่อมบำรุงตามรอบปกติ → `normal`
- Layer 3 (Disruptive / Public Inconvenience):** ความเดือดร้อนในชุมชน เช่น ท่อแตกน้ำไม่ไหล ไฟดับทั้งตำบล ขยะส่งกลิ่นเหม็น → `medium`
- Layer 4 & 5 (Keyword & Category Baseline):** แมตช์คำสำคัญทางความหมาย และ Fallback ตามหมวดหมู่

จุดเด่นเชิงวิศวกรรม

- Zero External Latency:** วิเคราะห์เสร็จสิ้นภายใน < 2 มิลลิวินาที (ไม่ต้องรอ Round-trip ไปยัง Claude/Gemini)
- Resilience Fallback:** ทำงานเป็นฐานรองรับทันทีหากไม่มี API Key หรือเมื่อเครือข่ายอินเทอร์เน็ตภายนอกขัดข้อง
- 100% Verified Accuracy:** ผ่านการทดสอบครอบคลุม 134 Few-shot test cases ในชุดทดสอบอัตโนมัติ (Group 7 ใน `scripts/runTests.js`)

6. เครื่องยนต์ประมวลผลทางธุรกิจ (ต่อ: Geo, ต้นทุน และใบงาน)

Core Logic

3. Geospatial & Duplicate Detection

การวิเคราะห์เชิงพื้นที่และตรวจจับเรื่องร้องเรียนซ้ำซ้อน:

- **Haversine Formula:** คำนวณระยะห่างระหว่างจุดพิกัด 2 จุดบนผิวโลก หากพิกัดอยู่ในรัศมี < 100 เมตร และเป็นหมวดหมู่เดียวกัน ระบบจะแจ้งเตือนว่าอาจเป็นซ้ำซ้อน
- **Admin Duplicate Merge:** Admin สามารถสังรวมตัว (Merge) ได้ โดยตัวรองจะถูกเปลี่ยนสถานะเป็น `merged` คะแนนโหวตและผู้ติดตามจะถูกถ่ายโอนไปยังตัวหลักอัตโนมัติ

4. Cost Tracking & District Analytics

การเงินและสถิติเชิงพื้นที่สำหรับผู้บริหาร:

- **Material Auto-Computation:** ช่างบันทึกรายการวัสดุ ระบบจะคูณ `quantity × unitPrice` และสรุป `totalRepairCost` รวมให้อัตโนมัติ (จำกัดจำนวนไม่เกิน 100 ชิ้นต่อรายการ)
- **District Extraction:** สกัดชื่อเขตและแขวงจากข้อความที่อยู่ด้วย Regular Expression ลงฟิลด์ `district` อัตโนมัติเพื่อนำไปสรุปงบประมาณและสถิติใน CEO Dashboard

5. Digital Work Order & Signature Engine (ใบสั่งงานราชการพร้อมลายเซ็นดิจิทัล)

เพื่อความโปร่งใสและตรวจสอบย้อนหลังได้ เพื่อช่างซ่อมงานเสร็จสิ้น ประชาชนหรือผู้ตรวจรับงานสามารถลงลายเซ็นดิจิทัลผ่าน HTML5 Canvas บนหน้าจอโทรศัพท์หรือแท็บเล็ต ลายเซ็นจะถูกแปลงเป็น Base64 PNG Data URL และบันทึกลงใน `ticket.workOrder` พร้อมชื่อผู้เซ็นและวันเวลา สามารถพิมพ์หรือดาวน์โหลดใบงานราชการฉบับสมบูรณ์ได้ทันทีผ่าน `GET /api/tickets/:id/work-order`



7. แค็ตตาล็อก REST API และ WebSocket Protocol

API & Real-time

ResolveNow ให้บริการ RESTful API ที่มีมาตรฐานความปลอดภัยสูง ทุก Endpoint มีการตรวจสอบสิทธิ์ (Authentication & Role-based Authorization) พร้อมทั้ง Real-time Event Synchronization ผ่าน Socket.IO

1. Authentication & Gate Management (/api/auth)

Method	Endpoint	Access	Description & Payload
POST	/send-otp	Public	ส่งรหัส OTP 6 หลักทางอีเมล (Rate limited 30 req/15m): { email }
POST	/register	Public	ตรวจสอบ OTP และเปิดบัญชีประชาชน: { firstName, lastName, email, password, otp }
POST	/login	Public	เข้าสู่ระบบด้วย bcrypt verification สร้าง Session: { email, password }
POST	/gate-verify	Public	ตรวจสอบ Passcode บันทึกสิทธิ์ลง Session (จำกัด 5 ครั้ง/15m): { passcode, portal }
GET	/gate-status	Public	ตรวจสอบสถานะการปลดล็อค Gate ของ Portal ในเซสชันปัจจุบัน: ? portal=admin
POST	/gate-lock	Public	สั่งล็อค Gate ใน Session ทันทีเมื่อผู้ใช้กดออกจากหน้าจอ: { portal }
POST	/test-email	Admin	เครื่องมือทดสอบการส่งอีเมล 3 ชั้น วัด Latency และระบุ Provider ที่จัดส่งสำเร็จ
GET	/mail-status	Admin	ตรวจสอบสถานะ Credentials ของ Gmail SMTP, SendGrid และ Resend

2. Ticket Lifecycle (/api/tickets) – หมวดจัดการงาน

Method	Endpoint	Access	Description & Payload
GET	/	Auth	ดึงรายการตั๋ว: citizen (ของตนเอง), tech (ตามสังกัดหรือที่รับงาน), admin (ทั้งหมด)
POST	/	Citizen	สร้างเรื่องร้องเรียนใหม่ แบบรูปได้สูงสุด 5 รูป กำหนด SLA และส่ง Flex เข้า LINE
PUT	/:id/status	Tech/Admin	เปลี่ยนสถานะตั๋วตาม State Machine (ตรวจ SLA Breach อัตโนมัติ): { status }
PUT	/:id/assign	Admin	มอบหมายงานให้ช่าง (assignedTo) พร้อมเปลี่ยนสถานะเป็น assigned
POST	/:id/upload/before	Technician	อัปโหลดภาพถ่ายก่อนเริ่มปฏิบัติงานซ่อมบำรุง
POST	/:id/upload/after	Technician	อัปโหลดภาพถ่ายหลังปฏิบัติงานเสร็จสิ้น (แบบได้สูงสุด 5 ภาพ)

7. แค็ตตาล็อก REST API (ต่อ: SLA, วัสดุ และ WebSockets)

API & Real-time

2. Ticket Lifecycle (ต่อ: SLA Pause, วัสดุ และใบงาน)

Method	Endpoint	Access	Description & Payload
POST	/:id/materials	Tech/Admin	บันทึกรายการวัสดุและค่าใช้จ่ายซ่อมบำรุง: { materials: [{ name, quantity, unitPrice }] }
POST	/:id/sla/request-pause	Tech/Admin	ส่งคำขอพักเวลา SLA ชั่วคราวเมื่อรออะไร: { reason }
POST	/:id/sla/approve-pause	Admin	Admin อนุมัติการพักเวลา SLA และบันทึกเวลา Freeze ลงในระบบ
POST	/:id/sla/resume	Tech/Admin	ยกเลิกการพักเวลา และขยาย Deadlines ชดเชยเวลาที่ถูกพักจริง
POST	/:id/merge	Admin	รวมตัวที่ซ้ำซ้อนเข้ากับตัวหลัก ถ่ายโอนคะแนนโหวตและผู้ติดตาม
POST	/:id/work-order/sign	Tech/Admin	บันทึกลายเซ็นดิจิทัลผู้ตรวจรับงาน: { signedByName, signatureData, notes }
GET	/:id/work-order	Public/Auth	แสดงหน้าเอกสารใบสั่งงานราชการดิจิทัลพร้อมพิมพ์
DELETE	/	Admin	ลบตัวทั้งหมดในระบบ ต้องใช้ adminPassword จาก ADMIN_DELETE_PASSWORD

⚡ Socket.IO Real-time Events Protocol

Event Name	Direction	Payload & Security Validation
dm_join	Client → Server	เข้าร่วมห้องแชท DM ป้องกัน IDOR: ตรวจสอบว่า data.userId === session.userId หากเป็น Admin เข้า admin_dm หากเป็น Citizen เข้า citizen_dm_<id>
dm_message	Server → Client	ยิงข้อความแชทตรงแบบเรียลไทม์ไปยังห้องของผู้รับ พร้อม Push Notification
ticket_updated	Server → Client (Broadcast)	ยิงแจ้งเตือนทุกฟอร์ทิลเมื่อมีการเปลี่ยนสถานะ: มอบหมายงาน หรือพักเวลา SLA
ping_heartbeat	Client → Server	ตรวจสอบการเชื่อมต่อ Socket มีชีวิตอยู่ ตอบกลับด้วย pong_heartbeat กันที

8. ชุดทดสอบอัตโนมัติ (Automated Test Suite & QA)

Quality Assurance

ResolveNow มีชุดทดสอบอัตโนมัติแบบครบวงจร (Automated Verification Test Suite) ที่รันผ่านคำสั่ง `npm test` หรือ `node scripts/runTests.js` ครอบคลุม **26 การทดสอบใน 7 กลุ่มงานหลัก** พร้อมการทดสอบความแม่นยำของโมเดลภาษาไทย 134 Cases:

**ผลการทดสอบล่าสุด (V18.0)**

สถานะ: **PASSED 100%**

จำนวนการทดสอบทั้งหมด: 26 Tests (Total 153 Assertions)

อัตราความสำเร็จ: 26 / 26 (100%)

ความเร็วในการทดสอบ: < 0.8 วินาที

**คำสั่งรันชุดทดสอบสำหรับ Developer**

```
# รันผ่าน npm script
npm test

# หรือรันผ่าน Node.js โดยตรง
node scripts/runTests.js
```

*ทุกครั้งที่มีการแก้ไขโค้ดใน `routes/`, `models/` หรือ `utils/` นักพัฒนาต้องรันคำสั่งนี้เพื่อยืนยันว่าไม่มีฟังก์ชันถดถอย (No Regression)

กลุ่มการทดสอบ (Group)	จำนวน Tests	ขอบเขตและเป้าหมายการทดสอบ (Verification Scope)
Group 1: SLA Engine & Breach Detection	9 Tests	ตรวจสอบกฎชั่วโมง SLA ทั้ง 3 ระดับ (Urgent 2h/8h, Med 8h/48h, Norm 24h/72h), ตรวจสอบการคำนวณวันสิ้นสุดล่วงหน้า, การตรวจจับทั้งหมดเวลาสำหรับสถานะ <code>pending</code> และ <code>in_progress</code> , และยืนยันว่าการพักเวลา (<code>paused</code>) จะแจ้งการตรวจ Breach อย่างถูกต้อง
Group 2: Geo Calculations & Duplicate Detection	3 Tests	ตรวจสอบความแม่นยำของสูตร Haversine Distance (จุดเดียวกันระยะทาง = 0m, จุดที่ห่างกัน < 100m ตรวจจับเป็นตัวซ้ำ, และจุดที่ห่างกัน > 1km ไม่ถูกนับเป็นตัวซ้ำ)
Group 3: Security & XSS Sanitization	2 Tests	ยืนยันว่าโมดูล XSS สามารถตัดและแปลงแก้ภัยอันตราย เช่น <code><script>alert(1)</script></code> และ Handler ร้ายแรงอย่าง <code></code> ออกจากข้อความภาษาไทยได้อย่างสมบูรณ์
Group 4: PDPA & Privacy Protection	1 Test	ตรวจสอบการแปลงชื่อผู้ใช้ใน Audit Timeline ของระบบค้นหาสาธารณะ (Public Tracker) ให้เป็น <code>ประชาชนผู้แจ้ง</code> และข้างเป็นชื่อต้น เพื่อรักษาความลับตาม พ.ร.บ.คุ้มครองข้อมูลส่วนบุคคล
Group 5: Financial & Budget Aggregations	2 Tests	ตรวจสอบการคำนวณสัดส่วนผลต่างงบประมาณประจำเดือน (Δ%) และการจัดการกรณีเดือนก่อนหน้าเป็น 0 บาท โดยไม่เกิดข้อผิดพลาด Divide-by-Zero
Group 6: Database Model Schemas & Indexes	4 Tests	ตรวจสอบความสมบูรณ์ของ Mongoose Model Schemas และดัชนี (Indexes) สำคัญ เช่น <code>role</code> ใน User, <code>status</code> และ <code>ticketId</code> ใน HelpRequest, Compound Index ใน DirectMessage และ Geo/SLA Indexes ใน Ticket
Group 7: Cognitive Thai NLP Heuristics Engine	5 Tests (134 Cases)	ทดสอบฟังก์ชัน <code>ruleBasedUrgency()</code> เทียบกับชุดข้อมูลภาษาไทย 134 Few-shot test cases ครอบคลุม 7 หมวดหมู่ปัญหา การแยกแยะสัตว์มีพิษในบ้าน/นอกบ้าน การตรวจจับคำปฏิเสธ และการแยกแยะความขยะออกจากเพลงไห้มีขนาดใหญ่ โดยได้คะแนนความแม่นยำ 134/134 (100%)

9. การติดตั้ง, ตัวแปรระบบ และบัญชีทดสอบเริ่มต้น

Deployment & Setup

ResolveNow รองรับการรันทั้งบน Local Development Environment และ Cloud Production Platforms (เช่น Render, Heroku, AWS หรือ VPS ทั่วไป) โดยมีขั้นตอนการตั้งค่าที่ง่ายดาย:

บัญชีทดสอบเริ่มต้นของระบบ (Default Seeded Accounts – config/seed.js)

บทบาท (Role)	อีเมล (Email)	รหัสผ่าน (Password)	สังกัด / แผนกความเชี่ยวชาญ	Gate Passcode
ADMIN	admin@resolvenow.th	admin1234	ศูนย์สั่งการและผู้ดูแลระบบส่วนกลาง (ควบคุมทุกพอร์ทัล)	@Teng11421142
TECH 1 – 7	tech1@resolvenow.th ถึง tech7@...	tech1234	7 หมวด: ถนน (Road), ประปา (Water), ไฟฟ้า (Electricity), ขยะ (Garbage), สัตว์ (Animal), ต้นไม้ (Tree), ภัยพิบัติ (Hazard)	@Teng11421142
CITIZEN (DEV)	tenginpb@gmail.com	123456	ประชาชนทดสอบระบบ (สร้างตัว, แยก DM, ไร้ตัวตน)	ไม่ต้องใช้ Gate

ตัวแปรสภาพแวดล้อมที่สำคัญ (Environment Variables Specification – .env)

```
# Server & Environment
PORT=3001
BASE_URL=https://resolvenow-hlv5.onrender.com
NODE_ENV=development

# Database & Session Storage
MONGODB_URI=mongodb+srv://<user>:<pass>@cluster.mongodb.net/resolvenow
SESSION_SECRET=your-strong-random-session-secret-32-chars

# Security Master Password (ลบตัวทั้งหมด)
ADMIN_DELETE_PASSWORD=your-secure-admin-delete-password

# Email Providers (Triple-Provider Architecture)
MAIL_USER=your-email@gmail.com
MAIL_PASS=your-gmail-app-password-16-chars
SENDGRID_API_KEY=SG.your-sendgrid-api-key
RESEND_API_KEY=re_your-resend-api-key
```

```
# LINE Integration (Messaging API, Login, LIFF)
LINE_CHANNEL_TOKEN=your-line-channel-access-token
LINE_ADMIN_USER_ID=your-line-admin-user-id
LINE_LOGIN_CLIENT_ID=your-line-login-channel-id
LINE_LOGIN_CLIENT_SECRET=your-line-login-channel-secret
LINE_LOGIN_CALLBACK_URL=https://resolvenow-hlv5.onrender.com/auth/line/callback
LINE_LIFF_ID=your-line-liff-id

# Cloudinary Media Storage (รูปภาพตัว & Before/After)
CLOUDINARY_CLOUD_NAME=your-cloud-name
CLOUDINARY_API_KEY=your-api-key
CLOUDINARY_API_SECRET=your-api-secret

# Cognitive AI Models (Optional - มี 5-Layer Heuristics แล้ว)
ANTHROPIC_API_KEY=sk-ant-api03-...
GEMINI_API_KEY=your-gemini-api-key
```

คำสั่งเริ่มต้นและทดสอบระบบ (Quick Start Commands)

1. ติดตั้ง Dependencies

```
npm install
```

2. รันโหมด Development

```
npm run dev
```

3. ทดสอบระบบอัตโนมัติ

```
npm test
```

ResolveNow Developer Master Architecture & Engineering Manual – Version V18.0 (2026-09-18) Copyright © 2026 ResolveNow Team. All rights reserved.